

Chat On Steroids 小白指南

给 ChatGPT 装上"手脚和 Agent 小队"的开源桌面工具 —— 疑问式导读 + 动手实验

github.com/totec448-spec/chat-on-steroids

MIT License

≈4.2k Stars / 1144 commits

Electron + Chrome 扩展 + 本地 MCP

v2.1.25 (2026-10 克隆实测)

零 · 30 秒说清它是什么

一句话: Chat On Steroids 是一个开源桌面应用 + Chrome 扩展, 把 ChatGPT 网页版接到你电脑的本地 MCP 工具上——读文件、改代码、跑终端、控制浏览器和桌面、派多个 Agent 并行干活, 全部在 ChatGPT 聊天窗口里完成, 不用切到 Codex CLI。

它走红于日文社区, 传播口号是 "**GPT-6 Astra 额度白嫖 100 倍**"——这句话是营销话术, 真相见疑问 2。技术上它是个正经工程: 本地 MCP 服务器有四道安全门、文件访问有 realpath 沙箱、权限模型设计得相当讲究, 非常值得学习。

🔴 知识库关联: 本项目是 **[[MCP协议]]/[[MCP服务器]]** 的一个大型落地实例, 也是 **[[AgentHarness智能体玩具]]** 概念的又一形态——上次学的 AgentHarness 是"给模型套上执行循环", 这个项目是"给网页版 ChatGPT 套上本地执行能力"。

一 · 疑问清单 (本指南目录)

#	疑问	层次
1	"不用切到 Codex"到底什么意思？	是什么
2	"白嫖 100 倍"是真的吗？安全吗？	避坑
3	网页里的 ChatGPT 怎么摸到我硬盘上的文件？	架构
4	只绑 127.0.0.1 还不够？为什么要有四道安全门？	安全
5	工具面 Tool Surface 是什么？为什么拆两个连接器？	设计
6	怎么保证 AI 只碰批准过的文件夹？	沙箱
7	权限撤销后工具为什么"还在但不可用"？	设计
8	Goal 和 Loop 是什么？怎么让它自己连续干活？	自动化
9	对话太长爆上下文了怎么办？Compact & Resume	续命
10	Workers 多 Agent 和 code-mode 是怎么并行的？	进阶
11	这个项目给我什么能力？我能用它干嘛？	收获
12	动手实验：测绘仓库·复现安全门·复现权限模型	实验

二·逐条图文解答

Q1 "不用切到 Codex"是什么意思？

ChatGPT 网页版本没有"读你电脑文件、跑你终端命令"的能力。以前想让 AI 替你干本地活，得装 OpenAI 的 **Codex CLI**——换一套界面、换一套额度计费。这个项目的思路是：**别换工具，给 ChatGPT 网页版"长出手脚"**。

做法：本地跑一个 **MCP 服务器**（MCP = Model Context Protocol，模型上下文协议，工具调用的行业标准，见 [[MCP协议]]），暴露 read / apply_patch / exec_command 等工具；ChatGPT 网页把它添加为"连接器/应用"后，模型就能像调用任何 MCP 工具一样调用你本地的文件系统和终端。

🔗 类比：[[AgentHarness智能体挽具]] 里学过"模型是大脑，harness 是手脚"。Claude Code 是 Anthropic 官方给 Claude 造的手脚；这个项目是社区给 ChatGPT 网页版造的手脚。

Q2 "白嫖 100 倍"是真的吗？是破解额度吗？

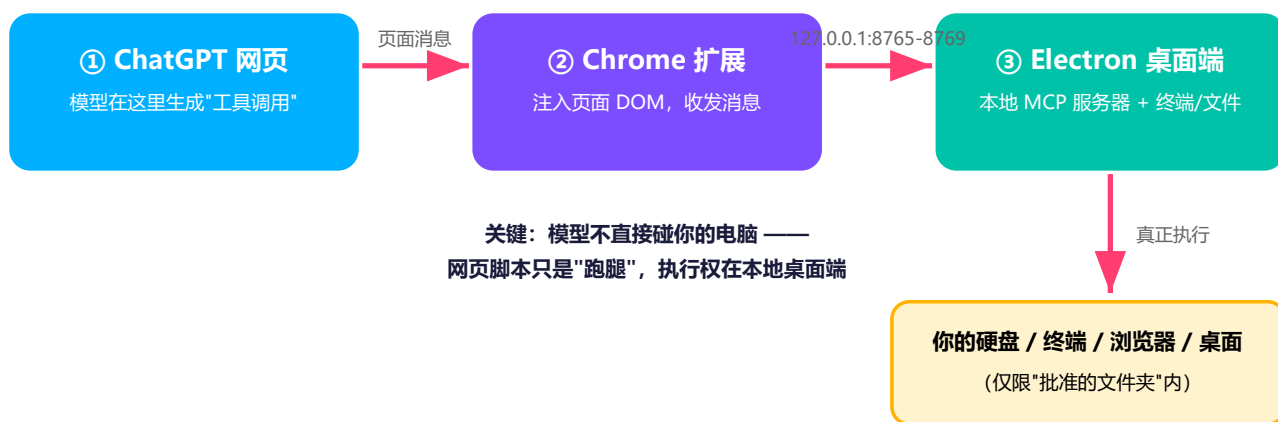
不是破解。README 原文明确声明："Workers、Goal/Loop、Compact & Resume 等功能 **do not grant extra quota or model access**（不提供额外额度或模型访问），不得用于规避速率限制、用量上限或账号限制。"

传播话术的真实含义：**ChatGPT 网页订阅的对话额度比 Codex 的 agent 用量额度宽松得多**，用前者跑 agent 任务，同样的订阅费能干多得多的活——这是"换一条已付费的通道"，不是"打开隐藏额度"。

⚠ **真实风险**（README 自己也警告了）：它通过 Chrome 扩展"观察并自动化 ChatGPT 浏览器界面"，这不是 OpenAI 官方公开的自动化 API，可能违反服务条款，存在警告、限流甚至封号风险；且与本仓库判例 55e30d0a / df617fa1 那种"无可克隆、无可动手的薅额度偏方"完全不同——那两个判的是平台口子，这个判学习类是因为它是真开源工程（MIT、可克隆、架构值得学）。

Q3 网页里的 ChatGPT 怎么摸到我硬盘上的文件？

靠三方接力，这是全项目最核心的架构图：



①你在 chatgpt.com 里说"帮我改这个项目的 bug"；②扩展从页面 DOM 里捕捉到消息，转发给本机端口；③桌面端里的 MCP 服务器执行工具，把结果原路送回，显示成聊天里的工具调用卡片。**ChatGPT 侧只是一个"会说话的调度员"**，所有危险操作都发生在你本机、被本地权限系统管着。

Q4 只绑 127.0.0.1 还不够？为什么要有四道安全门？

只监听本机回环地址，理论上外网摸不到。但浏览器里跑的恶意网页仍可能打进来——这就是 **DNS Rebinding (DNS 重绑定)** 攻击：攻击者让 evil.com 先解析到自己的服务器，把你的浏览器骗过去后，再把 DNS 记录切换成 127.0.0.1——你的浏览器就会“以为”在访问 evil.com，实际请求打进了本机端口。同源策略管不了它，因为脚本眼里这还是“自己的域名”。所以 server.ts 在处理任何请求前跑四道独立检查：

门	检查	挡什么
1. 秘密令牌路径	URL 必须带每次会话随机生成的 token 段，timingSafeEqual 比对	猜 URL 的扫描器、CSRF
2. Host 校验	Host 头必须是 loopback	DNS 重绑定
3. Origin 校验	若带 Origin (浏览器发起)，Origin 也必须 loopback	任意网页跨域驱动
4. 体积上限	body ≤ 8MB	恶意巨型包

```
server.ts 注释（原文）：
Bound to 127.0.0.1 on an ephemeral port, never 0.0.0.0, so nothing on the LAN can reach it.
Four independent checks run before any MCP handling.
```

实验 E2 会让你亲手把四道门复刻出来并用 5 种攻击视角实测（见疑问 12）。

Q5 工具面 (Tool Surface) 是什么？为什么拆两个连接器？

工具面 = 一个连接器对外公布的全部工具集合。项目拆成两个，各自独立发现、独立授权、各用一条带 token 的路径：

连接器	管什么	工具
Core (三平台必有)	批准的文件、补丁、终端、任务计划、子 Agent	read / view_image / find / apply_patch / exec_command / write_stdin / update_plan / agents
Desktop (Win/mac 可选)	屏幕、窗口、鼠标键盘、剪贴板	observe / computer

拆分的本质是“权限边界”：想让它读代码 ≠ 想让它动你的鼠标。不装 Desktop 连接器，模型就根本没有操作桌面的工具——比“有工具但提醒你小心”安全得多。有趣细节：update_plan 工具是从 OpenAI Codex 的同名实现 (Apache-2.0) 改编来的，plan-tool.ts 里注明了出处。

Q6 怎么保证 AI 只碰批准过的文件夹？

靠 **路径沙箱** (sandbox.ts)，设计哲学一句话：**"约束写在代码里，绝不写在提示词里"** (Enforcement lives here, in code. It is never delegated to prompt text.)

批准的根目录 F:\myproject\ (canonical 后)

/project/src/... ✓

/project/docs/... ✓

symlink → C:\Users\...
穿透后被 realpath 抓包 ✗

规则：fs.realpath 规范化真实路径 → 与规范化后的根目录做前缀比对
符号链接 / NTFS junction / reparse point 穿出去 = 直接拒绝

三个要点：① 模型看到的是**虚拟路径**（如 /project/src/main.ts），沙箱负责映射到真实路径；② 判定"在不在批准范围内"用的是 **fs.realpath 规范化后的真实路径**——在树里种一个指向外部的符号链接也逃不出去；③ 对 Windows 特意用原生 realpath 处理大小写不敏感和 WSL 路径别名。

Q7 权限撤销后，工具为什么"还在名单上但用不了"？

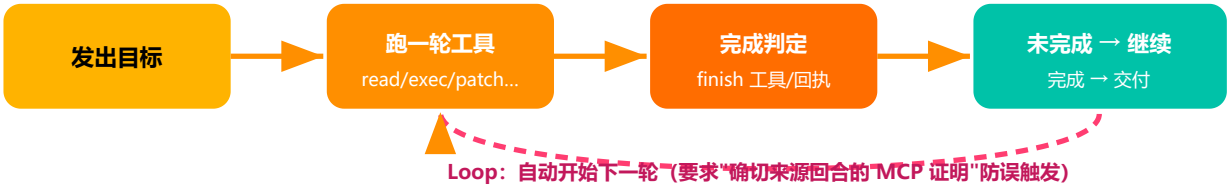
这是本项目最巧妙的设计之一：**工具暴露单调性**。ChatGPT 侧会**缓存工具快照**——如果端点运行中突然把某工具从名单里删掉，客户端缓存的快照就失效了，可能引发整段对话的工具列表错乱。所以：

✓ 规则：权限开启时工具才注册进快照（暴露面只增不减、单调）；权限中途撤销 → 工具名还留在快照里，但 handler 每次调用都实时校验当前权限，关闭时返回 TOOL_DISABLED。

另一个细节：每个工具带 **readOnlyHint** 注解。ChatGPT 把没有 readOnlyHint 的工具一律当写操作，每次调用都弹确认框——所以纯读工具（如 read）老老实实标 true，用户的确认弹窗就少了。注释原话："Annotations matter for real behaviour, not just documentation."

Q8 Goal 和 Loop 是什么？怎么让它自己连续干活？

普通聊天是“一问一答”，模型说完就停。**Goal** 给它一个目标让它自主完成；**Loop** 则是“这一轮结束后自动继续下一轮”，直到目标完成——相当于在 ChatGPT 网页外面套了一个**自动续杯的执行循环**（[[Agent循环]] 的思想：感知→思考→行动→再感知）。



防呆设计很细：docs 里写明“自动 Loop 必须有**exact source-turn MCP proof**（确凿的来源回合工具调用证据）”才允许触发，用户手动输入永远排在自动决策前面——**人的消息优先**。

Q9 对话太长爆上下文了怎么办？Compact & Resume（交接）

网页版 ChatGPT 的上下文有限，长任务干一半就“失忆”。方案：**handoff（交接简报）**——让当前对话把“做到哪了”写成一份**简报**存到本地，再开新对话时把简报灌回去继续干。与 [[Compaction上下文压缩算法]] 的不同：

	Compaction（Claude Code 等）	Handoff（本项目）
发生在哪	同一会话内，历史被压缩替换	跨会话 ：旧对话→本地文件→新对话
由谁写	系统压缩历史	模型自己把简报作为“最终回答”写出
防伪造	系统控制	交接 id 由本地 randomUUID 生成 ，绝不让模型自己编 id——注释说得很清楚：模型自造 id 会撞真 id、覆盖别人的简报

交接时还会附带**任务计划快照**，并诚实地标注“这是 reported progress, not verification evidence”（汇报的进度，不是验证过的证据）——连措辞都在防“假完成”（可关联 [[AI味模式库与信号非证据]] 的思想）。

Q10 Workers 多 Agent 和 code-mode 是怎么并行的？

Workers (agents 工具)：主对话可以派出多个子 Agent 并行干活，各自保留上下文；有整套"找回"机制——子 Agent 沉默太久 (silent ceiling) 会触发恢复、会话状态有 reply ledger 记账、身份对不上 (IdentityLost) 要重新认领。**多 Agent 最大的工程难题不是"派活"而是"收尸"**，这个项目的 recovery 代码占比相当大。

code-mode (exec 工具)：给模型一个受限 JavaScript 运行时，让它在一段代码里编排多个工具调用 (await tools.read(...)、Promise.all 并发)，**中间结果留在运行时里不进上下文，只有最终 text/image 输出才给模型看**——大幅省 token 和往返次数。限制清单也很硬核：64k 源码、32MiB 内存、2s CPU、60s 总时长、32 次调用、无 Node/文件系统/网络。合同对标 OpenAI Codex 的 code-mode 协议。

三 · 这个项目给我什么（能力清单）

能力	对你意味着什么
 学一套"网页模型 × 本地执行"的完整架构	看懂 Agent 类产品的通用骨架：模型调度、本地执行、权限隔离
 抄得走的本地服务安全范式	四道门 (token 路径 / Host / Origin / 体积上限) 可直接套用到你自己写的任何本地 HTTP 服务
 realpath 沙箱模式	任何"AI 改文件"类工具的防逃逸标准答案
 MCP 连接器工程参考	工具注解、单调暴露、权限实时校验——做自己的 MCP 服务器时全是现成作业
 多 Agent 编排的"收尸"清单	沉默恢复、身份认领、回执记账——自己写多 Agent 前先看它踩过的坑

 **使用边界**：真要用它连 ChatGPT，请先想清楚浏览器自动化违反 ToS 的封号风险；学习其架构不需要登录任何账号（本指南实验全部零账号零依赖）。

四 · 动手实验（已全部真实运行验证）

实验总览：3 个零依赖 Node 实验，不登录任何账号，只跟源码和协议打交道。 环境：Node v24 (v18+ 即可)。材料在 `exercise\` 目录。

实验 E1 • 仓库测绘 (11 项断言)

操作： `node e1_repo_survey.mjs ../repo`

目的： 不启动 Electron 应用，静态扫描源码验证文档声称的功能真实存在。

检测： 11 项断言全 PASS，未行 `===` 结果：11 PASS / 0 FAIL `===`

真实输出 (节选)： 四道门全部落地 true；readOnlyHint 7 处；扩展白名单 127.0.0.1:8765-8769；docs/ 下 74 篇 worklog。

坑： 初版脚本 8/10——`update_plan` 不在 `tools-core.ts` 而在独立 `plan-tool.ts` (自 Codex Apache-2.0 改编)，`goal/agent` 文件主要在 `src/shared/`。断言要跟着架构走。

实验 E2 • 复现 MCP 端点安全四道门 (6 次请求实测)

操作： `node e2_mcp_gates.mjs`

目的： 亲手复刻 `server.ts` 的四道门，从 5 个攻击视角实测拦截效果。

真实输出： 错误 token→404；恶意 Host(DNS rebinding)→403；外站 Origin→403；超 body→413；token 错但 Host 对→404；合法请求→200。

坑 (重要)： 首跑攻击2 返回 200！原因：**undici (Node 内置 fetch) 会把自定义 host 头当"禁止头"剥掉**，导致根本没伪造成功。改用 `node:http` 原生 `request` 才能真实伪造 Host 头——这本身就是一课：测安全逻辑要用底层 API。

实验 E3 • 复现工具暴露单调性 (7 项断言)

操作： `node e3_monotonic_tools.mjs`

目的： 用 60 行代码模拟"快照只增不减 + handler 实时校验 + readOnlyHint"三件套。

真实输出： 7/7 PASS——撤销 `apply_patch` 权限后，工具仍在快照里 (保护客户端缓存)，但实际调用返回 `TOOL_DISABLED`。

✅ **预期结果：** 三个脚本在仓库根目录下依次运行，合计 24 项断言全部 PASS。任何一项 FAIL 都说明你对机制的理解和源码有出入——回去重读对应源文件。

五 • 一页带走

- ① **"白嫖 100 倍"是通道切换的营销话术，不是破解额度**——但它自动化网页 UI 确有 ToS 风险，学习架构无需承担该风险。
- ② 架构记忆图：**ChatGPT 网页（脑）→ Chrome 扩展（神经）→ 本地 Electron/MCP（手脚）**，执行权和权限全在本地。
- ③ 本地服务安全四件套：**token 路径 + Host 校验 + Origin 校验 + 体积上限**，防的重绑定攻击值得每个写本地服务的人记住。
- ④ 沙箱哲学一句话：**约束写在代码里，绝不写在提示词里**；判定用 realpath 规范化后的真实路径。
- ⑤ 好设计是"防御性"的：**单调工具暴露护缓存、readOnlyHint 省确认、handoff id 防伪造、Loop 要 MCP 证据**——处处假设会出错。